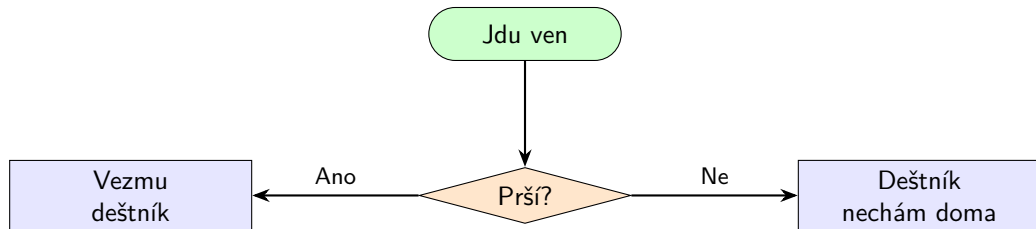


Lekce 03: Podmínky

Úvod do Pythonu na Micro:bitu

Rozhodování podle podmínek děláme pořád:



Podmínka v Pythonu: if / else

```
if podmínka:  
    # co se stane, když je podmínka splněna  
else:  
    # co se stane jinak
```

- ▶ Za if a else musí být **dvojtečka**
- ▶ Vnitřní příkazy musí být **odsazený** (Tab nebo 4 mezery)
- ▶ Odsazení = součást jazyka — Python ho vyžaduje

Podmínka v Pythonu: if / else

```
if podmínka:  
    # co se stane, když je podmínka splněna  
else:  
    # co se stane jinak
```

- ▶ Za if a else musí být **dvojtečka**
- ▶ Vnitřní příkazy musí být **odsazený** (Tab nebo 4 mezery)
- ▶ Odsazení = součást jazyka — Python ho vyžaduje

Nejčastější chyba

Zapomenutá dvojtečka nebo špatné odsazení → `IndentationError`.
Thonny odsazuje automaticky po dvojtečce — nechej ho pracovat.

True a False

Podmínka musí být buď **True** (pravda) nebo **False** (nepravda).

Tlačítko na micro:bitu vrací právě tyto hodnoty:

```
button_a.is_pressed()    # True, když je stisknuto  
                          # False, když není
```

True (pravda)

Podmínka je splněna.
Provede se blok za `if`.

False (nepravda)

Podmínka není splněna.
Provede se blok za `else`.

while True: — program, který běží pořád

Bez smyčky by se tlačítko zkontrolovalo jen jednou.

Přidáme while True: — program se **opakuje donekonečna**:

```
from microbit import *  
  
while True:  
    if button_a.is_pressed():  
        display.show(Image.HEART)  
    else:  
        display.show(Image.SAD)
```

while True: je speciální případ cyklu — podrobně v příští lekci.

Zastavení: tlačítko **Stop** v Thonny nebo **Reset** na micro:bitu.

Více větví: if / elif / else

elif = „jinak jestliže” — přidáme tolik větví, kolik potřebujeme:

```
while True:
    if button_a.is_pressed():
        display.show(Image.HAPPY)
    elif button_b.is_pressed():
        display.show(Image.ANGRY)
    else:
        display.show(Image.ASLEEP)
```

Python testuje podmínky **odshora dolů** a provede první, která platí. Zbytek přeskočí.

Obě tlačítka najednou: and

Klíčové slovo **and** spojí dvě podmínky — obě musí platit:

```
while True:
    if button_a.is_pressed() and button_b.is_pressed():
        display.scroll("Obe!")
    elif button_a.is_pressed():
        display.show(Image.ARROW_W)
    elif button_b.is_pressed():
        display.show(Image.ARROW_E)
    else:
        display.show(Image.DIAMOND_SMALL)
```


Obě tlačítka najednou: and

Klíčové slovo **and** spojí dvě podmínky — obě musí platit:

```
while True:
    if button_a.is_pressed() and button_b.is_pressed():
        display.scroll("Obě!")
    elif button_a.is_pressed():
        display.show(Image.ARROW_W)
    elif button_b.is_pressed():
        display.show(Image.ARROW_E)
    else:
        display.show(Image.DIAMOND_SMALL)
```

Proč musí být **and** jako první?

Kdybychom dali **and** větev na konec, podmínka `button_a.is_pressed()` by ji předběhla a nikdy bychom se tam nedostali.

- ✓ `if podminka:` — provede blok, je-li podmínka `True`
- ✓ `else:` — provede se, když podmínka neplatí
- ✓ `elif podminka:` — další větev podmínky
- ✓ `True / False` — výsledek podmínky
- ✓ `button_a.is_pressed()` — vrátí `True` nebo `False`
- ✓ `and` — obě podmínky musí platit zároveň
- ✓ `while True:` — program běží a kontroluje donekonečna
- ✓ Odsazení a dvojtečka jsou povinné — Python je vyžaduje